new relic

Ingestion Team

AI-Led Development Environment

Claude Code + GitHub Copilot

- 🚀 5× faster feature delivery
- 🧠 Day-1 onboarding via /bootstrap
- 🔍 30-second NR incident diagnosis
- ⚡ Multi-agent parallel work with /avengers
- 📦 Replayable golden patterns — knowledge that compounds

20 Skills

auto-trigger on file type

28 CRG tools

code-review-graph MCP

12+ NR MCP tools

live observability

Memory + Goldens

knowledge that compounds

Budget dial

🟢🟡🔴 daily spend cap

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

The Problem — Four Pains We Were Living With Every Week

🕐 Onboarding: 2+ Weeks

New engineers spent 2 weeks asking Slack, grepping git history, and re-deriving patterns that lived in people's heads.

Cost: ~4 eng-weeks/year per team

🔍 Incident Diagnosis: 2–3 Hours

Alert fires at 2am → manual NR UI hunt → log grep → Slack teammates → hypothesis → repeat.

Cost: ~12h/year MTTR overhead (before fix time)

💥 Blast Radius: Invisible

'1-day fix' became a 5-day incident because nobody knew what else would break before a line was written.

Cost: ~1 sprint/quarter absorbed in rework

🗂️ Architecture: In People's Heads

Knowledge in Slack threads and the senior engineer's memory. Every off-boarding = weeks of lost context.

Cost: exponential as team grows

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Our Answer: Two AI Tools, One Team — Each Does What It Does Best

GitHub Copilot

- ✓ Line & function autocomplete in IDE
- ✓ Inline chat: explain this function
- ✓ Generate boilerplate fast
- ✓ Test skeleton generation
- ✓ Rename / refactor suggestions

- × No codebase memory across sessions
- × Can't query NR, Jira, Confluence
- × No blast-radius analysis
- × No multi-agent parallelism

→ Together

Claude Code (our environment)

- ✓ Investigation: queries NR, Jira, Confluence live
- ✓ Memory: hot.md loads full context at session start
- ✓ Blast radius: get_impact_radius before writing code
- ✓ /avengers: 5 parallel agents on one task
- ✓ /golden + /replay: compounding pattern library

- ✓ 20 skills auto-trigger on file path / topic
- ✓ Model routing: Haiku → Sonnet → Opus by task
- ✓ Budget dial: 🟢🟡🔴 daily spend awareness
- ✓ Hooks: self-heal, cost record, MCP token refresh

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Three-Layer Architecture — Global → Project → Repo

GLOBAL ~/.claude/

```
CLAUDE.md      ← rules: security, model routing, workflows
budget.json    ← daily=$50  weekly=$120  monthly=$300
golden/        ← reusable session patterns + index.json
skills/        ← 20 skills (lang + domain + meta workflow)
hooks/         ← rtk, statusline, self-heal, cost-record
telemetry/     ← cost.jsonl  greps.jsonl
```

PROJECT ~/.claude/projects/<REPO_NAME>/

```
memory/hot.md           ← AUTO-LOADED ~2k tokens (session start)
memory/lessons.md       ← patterns, wins, anti-patterns
memory/architecture.md  ← derived from GRAPH_REPORT.md
memory/history.md       ← append-only session log
graphs/GRAPH_REPORT.md  ← AUTO-LOADED CRG summary + MCP hints
plans/                  ← mirrored from ~/.claude/plans/ at /wrap-up
```

REPO <repo>/

```
.claude/CLAUDE.md  ← only 2 @ refs → hot.md + GRAPH_REPORT.md (~4k total)
.mcp.json          ← memory-server + code-review-graph MCP
.code-review-graph/ ← SQLite AST graph (auto-updates on Edit)
.gitignore         ← .claude/, .mcp.json, .code-review-graph/
```

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Session-start token cost: Before = 10–30k tokens (5 @-loaded files) → After = ~4k tokens (hot.md + GRAPH_REPORT.md) — 3–7× cheaper

5 Minutes to a Fully-Wired Team Environment

```
# ── Install (one-time, per engineer) ──────────────────────────
$ git clone git@github.com:nr-mdakilahmed/claude_code_setup.git ~/claude_code_setup
$ cd ~/claude_code_setup && ./install.sh

→ Checks: claude  python3  git  jq  pipx  uv  (fatal if missing)
→ Installs code-review-graph via pipx  (Tree-sitter AST + 28 MCP tools)
→ Syncs memory-server  (pull-on-demand project memory via uvx)
→ Copies 20 skills to ~/.claude/skills/
→ Copies hooks: rtk, statusline, self-heal, cost-record, MCP refresh
→ Merges settings.template.json into ~/.claude/settings.json
→ Seeds ~/.claude/budget.json  +  golden/index.json

$ ./verify.sh          # end-to-end check: prereqs + skills + hooks + MCPs

# ── First visit to a repo ────────────────────────────────────
$ cd ~/Documents/GitHub/om-airflow-dags
$ claude                # opens Claude Code
> /bootstrap            # Opus: detect stack → seed memory → build graph → wire MCPs

# ── Every session end ────────────────────────────────────────
> /wrap-up              # persist history, lessons, refresh graph, regen hot.md
```

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

20 Skills — Context-Aware by Default, Explicit When You Need Them

META / WORKFLOW (7 — explicit /command only)

`/bootstrap`	First visit: detect stack, seed memory, build graph, wire MCPs
`/wrap-up`	Session end: history + todos + lessons + graph refresh + hot.md
`/avengers`	Opus orchestrates Sonnet specialists in parallel — for complex tasks
`/golden`	save | list | validate — capture reusable session templates
`/replay`	Load saved golden as prior-art for a matching task
`/budget`	status | set | override | report — daily/weekly/monthly spend
`/demo`	45-minute demo prep: problem-first, 5 acts, objection scripts

LANGUAGE / FRAMEWORK (7 — auto-trigger on file paths)

`/airflow`	`**/ dags/**/*.py`, `**/plugins/**/*.py`
`/pyspark`	`**/spark/**/*.py`, `**/*_spark.py`
`/python`	`**/*.py`, `pyproject.toml`, `requirements*.txt`
`/sql`	`**/*.sql`, `dbt models`, `schema.yml`
`/shell`	`**/*.sh`, `**/*.bash`
`/docker`	`**/Dockerfile*`, `**/docker-compose*.yml`
`/cicd`	`.github/workflows/*.yml`, `.pre-commit-config.yaml`

DOMAIN (6 — auto-trigger on topic keyword in prompt)

`/nrql`	NRQL / New Relic query
`/nralert`	alerts, muting rules, incidents
`/terraform`	`**/*.tf` or terraform mention
`/openmetadata`	ingestion yamls or OpenMetadata
`/mcp-builder`	MCP server / tool design
`/profiling`	cProfile / py-spy / slow Python

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

The Daily Habit — /bootstrap Once, /wrap-up Every Session

1 /bootstrap (once per repo) → 2 Open Claude Code in repo → 3 Work (skills auto-trigger) → 4 /wrap-up (session end)

/bootstrap phases (~15s):

```
Phase 1  detect-stack.sh → classify language + framework
Phase 2  seed-memory.sh  → 6 memory files + plans/ dir
Phase 3  write-project-claude.sh → .claude/CLAUDE.md + .mcp.json
Phase 4  build-graph.sh  → CRG install + build + GRAPH_REPORT.md
Phase 5  populate architecture.md from GRAPH_REPORT.md
```

/wrap-up phases (~10s):

```
Phase 0   corrections audit → lessons.md
Phase 1   append-history → new ## <date> block
Phase 2   todo update → Active/Backlog/Done
Phase 3   lessons dedupe → promote to CLAUDE.md
Phase 3.5 golden check (auto-prompt if 3 signals present)
Phase 4   code-review-graph update + regen GRAPH_REPORT.md
Phase 5   mirror-plans + regenerate hot.md
Phase 6   handoff summary with counts
```

hot.md is ~2k tokens (session start) · history.md, lessons.md, architecture.md are pull-on-demand via memory MCP · knowledge compounds across sessions

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Memory That Doesn't Decay — Knowledge Compounds Across Every Session

AUTO-LOADED

hot.md (~2k tokens)

Active todos (top 10) + recent lessons (last 10) + architecture summary.
/wrap-up regenerates it every session from the live memory files.

PULL ON DEMAND

lessons.md · architecture.md · history.md · todo.md · plans/

get_memory(topic) · search_memory(query) · list_lessons(tag) · get_todo(status) · recall_plan(slug)
Retrieved only when relevant — avoids drowning in stale context, 100–500 tokens/call.

REPLAYABLE

~/.claude/golden/<slug>.md (indexed in golden/index.json)

/golden save kafka-consumer-lag-fix → distills session into schema: Symptom · Root Cause · Steps That Worked · What NOT To Do
/replay kafka-consumer-lag-fix → validates file refs, loads as prior-art plan

Month 1: 8 blocking issues/task → Month 3: 1–2. Every session the system gets sharper, not staler.

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

NR MCP: 2-Hour Incident Hunt → 30-Minute Root Cause

BEFORE — Manual NR UI

```
# Alert fires 2am  →  you open NR UI

1. Navigate to entity manually
2. Logs tab → scroll → ctrl+F 'error'
3. Metrics tab → compare time ranges
4. Traces → click through spans
5. Slack teammates for context
6. Form hypothesis → test → repeat

⏱  avg: 2-3 hours to root cause
```

Lost: sleep, momentum, time to fix

→

AFTER — Claude Code + NR MCP

```
# Same alert, same 2am

> "analyze kafka consumer lag last 4h"

→ analyze_kafka_metrics(entity_guid)
  consumer_lag: 847,293  (↑ 12×)
  partition_balance: 3 of 8 skewed
  throughput: dropped 89% at 01:47

→ get_impact_radius('spark_ingest.py')
  callers: 2,  affected_flows: 4
  downstream: om_airflow_daily_dag

⏱  30 minutes to root cause + scoped fix
```

NR MCP tools: analyze_kafka_metrics · analyze_entity_logs · analyze_golden_metrics
list_recent_issues · get_distributed_trace_details · execute_nrql_query · list_entity_error_groups

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Code-Review-Graph: Blast Radius Before You Write a Line

code-review-graph — Tree-sitter AST + SQLite + 28 MCP tools per repo (auto-builds on /bootstrap, auto-updates on every Edit)

You edit spark_ingest.py

`get_impact_radius()`

Callers: 2
Flows: 4
Consumers: 1

Fix scoped.
Ship safely.

28 MCP tools — key ones:

`get_impact_radius`	Blast radius of a change — callers, callees, affected flows
`query_graph`	callers_of / callees_of / imports_of / tests_for / file_summary
`get_affected_flows`	Which execution paths pass through changed files
`get_review_context`	Token-efficient review: source + impact in one call
`detect_changes`	Risk-scored review: what changed and what it touches
`semantic_search_nodes`	Find functions/classes by name or semantic similarity
`get_architecture_overview`	High-level community structure + coupling analysis
`refactor_tool`	rename preview / dead_code scan / suggest misplaced functions

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

/avengers — Multi-Agent Orchestration for Tasks Too Big for One Pass

Nick Fury (Opus — Orchestrator)

Routes work · validates gates · blocks bad code · shuts down on completion

strange-architect
(design + API contracts)

stark-engineer-1...N
(Sonnet, parallel coders)

natasha-reviewer
(code + security review)

hawkeye-validator
(tests + blast radius)

Optional domain specialists (spawned on demand by Fury):

- rogers-data-engineer (pipelines, ETL, warehouse)
- maximoff-python-engineer (PySpark, Airflow, pandas)
- thor-devops (Terraform, CI/CD, K8s)

```
# Real example: Airflow DAG + dbt model + NR alert condition — what used to take 1 day
> /avengers 'add daily_reconciliation_dag with NR alert on SLA breach'
→ Fury reads spec + graph context
→ Spawns strange-architect (designs DAG topology) + stark-engineer-1 (writes DAG) in PARALLEL
→ stark-engineer-2 writes NR alert HCL while stark-engineer-1 finishes DAG
→ natasha-reviewer: code standards + OWASP + blast radius check
→ hawkeye-validator: runs tests, confirms 0 regressions
→ Fury: merges output, generates PR description, shuts down team
```

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Jira + Confluence MCP — Zero Copy-Paste, Every Session Documented

Dev merges PR → /wrap-up fires → Claude drafts → Jira updated → Confluence published

Jira — Auto-populated

Status: In Review → Done

Comment: what was done, decisions made

Time: logged from session duration

Linked: to commits via branch name

Jira MCP tools used:

`editJiraIssue` · `addCommentToJiraIssue`

`transitionJiraIssue` · `addWorklogToJiraIssue`

Confluence — Auto-published

Architecture decisions: auto-logged

Incident postmortem: page auto-created

Release notes: from git history

Categorized by area, files changed listed

Confluence MCP tools used:

`createConfluencePage` · `updateConfluencePage`

`searchConfluenceUsingCql` · `createFooterComment`

Zero manual effort. Every session documented. Every time.

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Model Routing — Right Tool, Right Cost, No Waste

Output cost dominates: Opus $75/M · Sonnet $15/M · Haiku $5/M. Default = Sonnet. Escalate per-turn, not per-session.

HAIKU 4.5 (cheapest)

- Jira / Confluence fetch + summarize
- NRQL query execution + format
- File search / grep / glob / cat
- Lint fix from explicit error
- Rename / format / simple transform
- Commit message from diff

SONNET 4.6 (DEFAULT)

- Bug fix with clear hypothesis
- Feature implementation (follows patterns)
- Unit tests on known function
- Cross-file refactor
- Code review on PR <500 lines
- Template-driven rewrite (skill rewrites)

OPUS 4.7 (5× cost — earn it)

- /bootstrap (deep codebase analysis)
- /avengers Fury (orchestrator)
- Architecture design + security review
- Production incident, unclear cause
- Complex PR review >500 lines critical path
- Multi-phase initiative planning

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Productivity Multipliers — Honest Numbers, Real DE Work

Repetitive DE tasks with /replay goldens

DAG edits, alert tuning, connector YAML, routine fixes

~50×

Novel feature work — graph-first nav + memory MCP

New pipeline, new data source, new alert policy

~10×

Architecture / design — thinking-bound, not context-bound

New service, schema design, cross-team dependencies

~3×

Prod debugging — your understanding is the bottleneck

Novel failure mode, unfamiliar system, on-call

~1×

Sustained team average: 15–25× over a month of real DE work

(100× aspirational — cherry-picked replay-heavy scenarios only)

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Before & After — Ingestion Team Metrics (3 Repos, ~3 Months)

Metric	Before (manual)	After (AI-led)
Engineer onboarding time	2+ weeks (Slack + git history)	**Hours — /bootstrap loads context day 1**
Incident MTTR (diagnosis)	2–3 hours (manual NR UI hunting)	**20–30 min (NR MCP structured output)**
Feature cycle time	14–20 days	**3–5 days**
Blast radius visibility	Unknown until prod breaks	**Instant — get_impact_radius pre-commit**
Rework from hidden deps	~1 sprint/quarter absorbed	**< 1 day (caught pre-commit)**
Context on new codebase	Days of grep + git archaeology	**Minutes — graph + hot.md loaded**
Security bugs caught	~40% (periodic manual review)	**80–100% (natasha-reviewer every PR)**
Test coverage	60–70%	**90% (hawkeye-validator gate)**

Repos: om-airflow-dags · spark-kafka-apps · tf-dataos-new-relic

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

AI Does the Work. Humans Stay in Control.

1	**Requirements approval**	Before planning begins — human writes the spec, Claude reads it
2	**Architecture plan approval**	Before any code is written — /bootstrap plan mode, human approves
3	**Security review sign-off**	natasha-reviewer flags OWASP findings — any critical BLOCKS pipeline
4	**Final PR merge**	Human reviews every PR — agent PRs treated identically to human PRs
5	**Production deployments**	Human initiates + monitors — agent creates release notes, human deploys

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

Getting Started — Week-by-Week Adoption Plan

Week 1 — Solo

- Install: ./install.sh + ./verify.sh (15 min)
- /bootstrap on your most-used repo
- Work normally — skills auto-trigger
- /wrap-up at every session end
- Check /budget status daily

Week 2 — Discipline

- All active repos bootstrapped
- Save first golden after a prod fix
- Try /replay on a recurring task
- Review grep-telemetry-report.sh weekly

Week 3 — Advanced

- /avengers on first real multi-file task
- Golden library: 3–5 patterns saved
- Tune model routing (Haiku for mechanical)
- Check: session start cost < $0.05

Week 4+ — Steady State

- 2–5 new goldens/week automatically
- Grep fallback rate < 10%
- lessons.md promoting to CLAUDE.md
- Team sharing golden library

🟢 **Green**	golden/*.md grows 2–5/wk · grep fallback <10% · session start <$0.05	🟡 **Yellow**	overrides ≥3/wk → raise caps · grep fallback >30% → tune graphignore	🔴 **Red**	engineers skip /wrap-up frequently · goldens never used → tags too narrow

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

Thank You

Questions?

Repo:	github.com/nr-mdakilahmed/claude_code_setup
Install:	git clone → ./install.sh → ./verify.sh
First visit:	/bootstrap | Every session end: /wrap-up